Pure Pursuit Controller

Implementation & Analysis

Rubesh

Implementation Details

Data Validation

- First, we prioritized **path and pose validation**. Before any control commands are calculated, we check that both path and pose data are available. This prevents errors if the data is delayed or temporarily unavailable, which can happen in dynamic environments.
- Prevents invalid data from being propagated down the program

```
// Check if robot's pose is available
if (std::isnan(rbt_x) || std::isnan(rbt_y) || std::isnan(rbt_h))
{
    RCLCPP_WARN(this->get_logger(), "Robot pose not available.");
    return;
}
```

Lookahead point selection

- A point selected at a specified distance ahead on the path as the target.
- A larger distance helps the robot move **more smoothly** but makes turns wider.
- A smaller distance gives tighter following but can make the movement jittery. Important to tune this balance.

Modular Callback Design

- We use specific callbacks for tasks like path updates and odometry, with a main timer loop for control logic.
- Generally separation of code in line with the **Single Responsibility Principle**
- This separation helps keep the code organized, making it easier to debug and develop further.

```
// Initialize topics for path and odometry
void initTopics()
{
    sub_path = this->create_subscription<nav_msgs::msg::Path>(
        "path", rclcpp::SensorDataQoS(),
        std::bind(&Controller::cbPath, this, std::placeholders::_1));
    sub_odom = this->create_subscription<nav_msgs::msg::Odometry>(
        "odom", rclcpp::SensorDataQoS(),
        std::bind(&Controller::cbOdom, this, std::placeholders::_1));
    pub_cmd_vel = this->create_publisher<geometry_msgs::msg::Twist>(
        "cmd_vel", rclcpp::SystemDefaultsQoS());
}
```

Limitations

Smooth Path Following

- The pure pursuit algorithm can cause the robot to oscillate, especially in narrow passages or when following paths with sharp turns.
- This oscillation leads to inefficient movement and can increase the risk of collisions.
- Solution: Dynamically adjusting the lookahead distance based on the robot's speed and the curvature of the path.

Kinematic Constraints

- We also implemented **velocity & acceleration constraints** to ensure the robot doesn’t exceed its physical capabilities.
- Without these constraints, the robot can overshoot or oscillate, especially on tight turns. By setting maximum linear and angular velocities and acceleration limits, we keep the robot’s movement stable and responsive.
- We also added a **stopping threshold** for goal achievement.
- This threshold helps the robot recognize when it’s close enough to the goal to stop, even if it hasn’t perfectly reached the exact location.